

Contents

1	Basis	1
1.1	Default Code	1
1.2	Some Useful Primes	1
1.3	偽隨機 & MT19937	1
1.4	快讀快寫	1
2	Data Structures	1
2.1	Disjoint Set (併查集)	1
2.2	Binary Indexed Tree	1
2.3	二維 BIT	1
2.4	Segment Tree	2
2.5	Treap	2
2.6	平板電腦	2
3	Mathematics	3
3.1	公式們	3
3.2	階乘 & 模逆元	3
3.3	GCD and LCM	3
3.4	EXGCD	3
3.5	Fast Power	3
3.6	矩陣乘法	3
3.7	FFT	3
3.8	質數表	4
3.9	歐拉函數	4
3.10	Miller-Rabin Algorithm	4
3.11	Pollard's Rho Algorithm	4
3.12	高斯消去法	4
3.13	約瑟夫問題	5
4	Graph	5
4.1	Topological Sort	5
4.2	Tarjan's Algorithm for BCC	5
4.3	Bellmen-Ford Algorithm	5
4.4	Dijkstra's Algorithm	5
4.5	Dinic's Algorithm for Maximum Flow	5
4.6	Dominator Tree	6
4.7	Floyd-Warshall Algorithm	6
4.8	Maximum Clique	6
4.9	Kosaraju's Algorithm for SCC	7
4.10	Kruskal's Algorithm for MST	7
4.11	Kuhn-Munkre Algorithm	7
4.12	Max-flow-min-cost	8
4.13	Kuhn Algorithm for Maching	8
5	Tree Algorithms	8
5.1	時間戳記 & 倍增表 (初始化)	8
5.2	LCA	8
5.3	Eular Tour for LCA	9
5.4	樹上差分	9
5.5	DSU on Tree	9
5.6	樹鏈剖分	9
6	String Algorithms	9
6.1	Suffix Array	9
6.2	Suffix Array (SAIS)	10
6.3	Suffix Automata	10
6.4	Z-algorithm	10
6.5	Knuth-Morris-Pratt Algorithm	10
6.6	Manacher's Algorithm	11
6.7	Minimum Rotation	11
6.8	Rolling Hash	11
6.9	Another Hash	11
7	Geometry	11
7.1	Basic Geometry Structure	11
7.2	Circle Cover Area	12
7.3	圓的公切線	12
7.4	圓的交點	12
7.5	Convex Hull	12
7.6	Convex Hull Tricks	13
7.7	Finding Farthest Point using CH	13
7.8	Pick's Theory	13
8	DP	13
8.1	區間 DP	13
8.2	位數 DP	13
8.3	SOS DP	14
9	Sorting	14
9.1	Merge Sort	14
9.2	Quick Sort	14
10	Miscellaneous	14
10.1	GNU Builtins	14
10.2	Discretization	14
10.3	Mo's Algorithm	14
10.4	Decimal	14
10.5	Fraction	14
10.6	Blessing	15
10.7	Graph Paper	16

1 Basis

1.1 Default Code

```
// g++args: -std=c++17 -Wall -Wshadow -fsanitize=undefined
#pragma GCC optimize("O3,unroll-loops")
#pragma target optimize("avx2,bmi,bmi2,lzcnt,popcnt")
#include <bits/stdc++.h>
#define ll long long
#define lll __int128
#define ld long double
#define pii pair<int, int>
#define D double
#define INF 0x7f7f7f7f7f7f7f7fLL
#define REP(i, n) for (int i = 0; i < n; i++)
#define PB push_back
#define SZ(x) (int)(x).size()
#define all(x) (x).begin(), (x).end()
using namespace std;
const int MXN = 2e6 + 5;
const int MOD = 1e9 + 7;
signed main() {
    ios_base::sync_with_stdio(0), cin.tie(0), cout.tie(0);
    return 0;
}
```

1.2 Some Useful Primes

```
/* 999983, 1097774749, 1076767633, 100102021, 999997771,
 * 1001010013, 1000512343, 987654361, 999991231,
 * 999888733, 98789101, 987777733, 999991921,
 * 1010101333, 1010102101, 1000000000039,
 * 1000000000000037, 2305843009213693951,
 * 4611686018427387847, 9223372036854775783,
 * 18446744073709551557 */
```

1.3 偽隨機 & MT19937

```
// chrono::steady_clock::now().time_since_epoch().count();
mt19937 gen(*seed*/ hash<string>("ShanC_orz"));
int randint(int a, int b) { // [a, b]
    return uniform_int_distribution<int>(a, b)(gen);
}
// srand(time(NULL)), rand() % (b - a + 1) + a;
```

1.4 快讀快寫

```
inline int read() {
    char c = getchar();
    int x = 0, f = 1;
    while (c < '0' || c > '9') {
        if (c == '-') f = -1;
        c = getchar();
    }
    while (c >= '0' && c <= '9')
        x = x * 10 + c - '0', c = getchar();
    return x * f;
}
inline void write(int x) {
    if (x < 0) putchar('-'), x = -x;
    if (x > 9) write(x / 10);
    putchar(x % 10 + '0');
}
```

2 Data Structures

2.1 Disjoint Set (併查集)

```
struct DSU {
    int n;
    vector<int> f, sz;
    DSU(int _n) : n(_n) { // 初始化
        f.resize(n);
        sz.resize(n);
        for (int i = 0; i < n; i++) f[i] = i, sz[i] = 1;
    }
    int find(int x) { // 返回 x 的根節點
        if (x == f[x]) return x;
        return f[x] = find(f[x]);
    }
    void merge(int x, int y) { // 將 x 和 y 合併
        x = find(x), y = find(y);
        if (x == y) return;
        if (sz[x] < sz[y]) swap(x, y); // 將小的併入大的
        sz[x] += sz[y];
        f[y] = x;
    }
};
```

2.2 Binary Indexed Tree

```
struct BIT {
    int n;
    vector<ll> bit;
    int lowbit(int x) { return x & -x; }
    BIT(int _n) : n(_n + 1) { bit = vector<ll>(_n + 1, 0); }
    void update(int x, ll v) { // 將 x 的值加上 v
        for (; x < n; x += lowbit(x)) bit[x] += v;
    }
    ll query(int x) { // 查詢區間 [1, x] 的總和
        ll ret = 0;
        for (; x; x -= lowbit(x)) ret += bit[x];
        return ret;
    }
    ll query(int l, int r) { // 查詢區間 [l, r] 的總和
        return query(r) - query(l - 1);
    }
    int kth(int k) {
        int x = 0;
        for (int i = 1 << __lg(n); i; i >= 1)
            if (x + i <= n && k >= bit[x + i - 1])
                x += i, k -= bit[x - 1];
        return x;
    }
};
```

2.3 二維 BIT

```
struct BIT {
    int n;
    vector<vector<ll>> bit;
    int lowbit(int x) { return x & -x; }
    BIT(int _n) : n(_n + 1) {
        bit.assign(n, vector<ll>(n, 0));
    }
    void update(int x, int y, ll val) { // 更新 (x, y)
        for (int i = x; i < n; i += lowbit(i))
            for (int j = y; j < n; j += lowbit(j))
```

```

        bit[i][j] += val;
    }
    ll query(int x, int y) { // 查詢 (1, 1) ~ (x, y) 的和
        ll ans = 0;
        for (int i = x; i; i -= lowbit(i))
            for (int j = y; j; j -= lowbit(j)) ans += bit[i][j];
        return ans;
    }
    ll range_query(int a, int b, int x, int y) {
        return query(x, y) - query(x, b - 1) -
            query(a - 1, y) + query(a - 1, b - 1);
    }
};

```

2.4 Segment Tree

```

#define cl (i << 1)
#define cr (i << 1 | 1)
#define NO_TAG 0
struct SegmentTree { // 1-base
    int n;
    vector<int> seg, tag;
    SegmentTree(int _n) : n(_n) { // 空的 SegmentTree
        seg.resize(n * 4), tag.resize(n * 4);
    }
    void push(int i, int l, int r) {
        if (tag[i] != NO_TAG) {
            seg[i] += tag[i]; // update by tag
            if (l != r)
                tag[cl] += tag[i], tag[cr] += tag[i]; // push
            tag[i] = 0;
        }
    }
    void pull(int i, int l, int r) {
        int mid = (l + r) >> 1;
        push(cl, l, mid), push(cr, mid + 1, r);
        seg[i] = max(seg[cl], seg[cr]); // pull (操作改這)
    }
    void build(int i, int l, int r) {
        if (l == r) {
            seg[i] = arr[l]; // 初始值
            return;
        }
        int mid = (l + r) >> 1;
        build(cl, l, mid), build(cr, mid + 1, r);
        pull(i, l, r);
    }
    void update(int i, int l, int r, int ql, int qr,
                int v) {
        push(i, l, r);
        if (ql <= l && r <= qr) {
            tag[i] += v;
            return;
        }
        int mid = (l + r) >> 1;
        if (ql <= mid) update(cl, l, mid, ql, qr, v);
        if (qr > mid) update(cr, mid + 1, r, ql, qr, v);
        pull(i, l, r);
    }
    int query(int i, int l, int r, int ql, int qr) {
        push(i, l, r);
        if (ql <= l && r <= qr) return seg[i];
        int mid = (l + r) >> 1, ans = -INF;
        if (ql <= mid) // (操作改這)
            ans = max(ans, query(cl, l, mid, ql, qr));
        if (qr > mid) // (操作改這)
            ans = max(ans, query(cr, mid + 1, r, ql, qr));
        return ans;
    }
    // 區間 [ql, qr] 加值 v
    void update(int ql, int qr, int v) {
        update(1, 1, n, ql, qr, v);
    }
    // 查詢區間 [ql, qr]
    int query(int ql, int qr) {
        return query(1, 1, n, ql, qr);
    }
};

```

2.5 Treap

```

struct Treap {
    int sz, val, pri, tag;

```

```

    Treap *l, *r;
    Treap(int _val) : sz(1), val(_val), tag(0) {
        l = r = NULL, pri = rand(); // 隨機產生優先度
    }
};
int size(Treap* a) { return a ? a->sz : 0; }
void pull(Treap* a) {
    a->sz = size(a->l) + size(a->r) + 1;
}
Treap* merge(Treap* a, Treap* b) {
    // val of a is always bigger than val of b
    if (!a || !b) return a ? a : b;
    if (a->pri > b->pri) {
        a->r = merge(a->r, b);
        pull(a);
        return a;
    } else {
        b->l = merge(a, b->l);
        pull(b);
        return b;
    }
}
void split(Treap* t, int k, Treap*& a, Treap*& b) {
    // a < k, b >= k
    if (!t) {
        a = b = NULL;
        return;
    }
    if (k <= t->val) {
        b = t;
        split(t->l, k, a, b->l);
        pull(b);
    } else {
        a = t;
        split(t->r, k, a->r, b);
        pull(a);
    }
}
Treap* add(Treap* t, int v) {
    Treap *val = new Treap(v), *l = NULL, *r = NULL;
    split(t, v, l, r);
    return merge(merge(l, val), r);
}
Treap* del(Treap* t, int v) {
    Treap *l, *mid, *r, *temp;
    split(t, v, l, temp);
    split(temp, v + 1, mid, r);
    return merge(l, r);
}
int position(Treap* t, int p) { // base 1
    if (size(t->l) + 1 == p) return t->val;
    if (size(t->l) < p)
        return position(t->r, p - size(t->l) - 1);
    return position(t->l, p);
}
int query(Treap* t, int k) { // num of >= k
    if (!t) return 0;
    if (t->val == k) return size(t->l) + 1;
    if (t->val > k) return query(t->l, k);
    return size(t->l) + 1 + query(t->r, k);
}

```

2.6 平板電腦

```

#include <bits/stdc++.h>

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/rope>
#include <functional>
#include <queue>
using namespace std;
using namespace __gnu_cxx;
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
            tree_order_statistics_node_update>
    set_t;
typedef cc_hash_table<int, int> umap_t;
typedef priority_queue<int> heap;

set_t s; // Insert some entries into s.
s.insert(12), s.insert(505);
// The order of the keys should be: 12, 505.
assert(*s.find_by_order(0) == 12);
assert(*s.find_by_order(3) == 505);

```

```
// The order of the keys should be: 12, 505.
assert(s.order_of_key(12) == 0);
assert(s.order_of_key(505) == 1);
s.erase(12); // Erase an entry.
// The order of the keys should be: 505.
assert(*s.find_by_order(0) == 505);
// The order of the keys should be: 505.
assert(s.order_of_key(505) == 0);
heap h1, h2;
h1.join(h2);
rope<char> r[2];
r[1] = r[0]; // persistenet
string t = "abc";
r[1].insert(0, t.c_str());
r[1].erase(1, 1);
cout << r[1].substr(0, 2);
```

3 Mathematics

3.1 公式們

$$\sum_{k=1}^n n = \frac{n(n+1)}{2}, \quad \sum_{k=1}^n n^2 = \frac{n(n+1)(2n+1)}{6}, \quad \sum_{k=1}^n n^3 = \left[\frac{n(n+1)}{2}\right]^2$$

$$\sum_{k=1}^n n^4 = n(1+n)(1+2n)(-1+3n+3n^2)/30$$

$$\sum_{k=1}^n n^5 = n^2(n+1)^2(2n^2+2n-1)/12$$

錯位排列

$$DP_i = \begin{cases} 0 \vee 1, & i = 1 \vee 2 \\ (i-1)(DP_{i-1} \times DP_{i-2}), & i \geq 3 \\ = iDP_{i-1} + (-1)^i & \text{另一種關係式} \end{cases}$$

生成樹數量 n^{n-2}

$$P_k^n = \frac{n!}{k!(n-k)!}, \quad C_k^n = \frac{n!}{(n-k)!}, \quad H_k^n = C_{n-1}^{n+k-1}$$

$$\pi = \arccos(-1), \quad e^{i\theta} = \cos \theta + i \sin \theta, \quad pV = nTR$$

$$a^{\varphi(m)} \equiv 1 \pmod m, \quad a^b \equiv a^{b \bmod \varphi(m)-1} \pmod m$$

$$\varphi(m) = m - 1, \text{When } m \text{ is Prime}$$

3.2 階乘 & 模逆元

```
ll fac[MXN], inv[MXN];
void init() {
    fac[0] = 1; // 0! = 1
    for (ll i = 1; i < MXN; i++)
        fac[i] = fac[i - 1] * i % MOD; // n! = n * (n-1)!
    inv[MXN - 1] = power(fac[MXN - 1], MOD - 2);
    for (ll i = MXN - 2; i >= 0; i--)
        inv[i] = inv[i + 1] * (i + 1) % MOD; // (n!)^(-1)
}
```

3.3 GCD & LCM

```
ll gcd(ll a, ll b) {
    if (b && a) // 輾轉先除法 -> 交叉取餘數直到不能再取
        while ((a %= b) && (b %= a)) {}
    return a + b;
}
ll lcm(ll a, ll b) { return a * b / gcd(a, b); }
```

3.4 EXGCD

```
ax + by = gcd(a, b)

int exgcd(int a, int b, ll& x, ll& y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    int now = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return now;
}
```

3.5 Fast Power

```
ll power(ll x, ll y, ll mod) {
    ll ret = 1;
    while (y) {
        if (y & 1) ret = ret * x % mod;
        x = x * x % mod, y >>= 1;
    }
    return ret;
}
```

3.6 矩陣乘法

```
struct Mat { // Mat t(r, c);
    ll a[200][200], r, c;
    Mat(int _r, int _c) : r(_r), c(_c) {
        memset(a, 0, sizeof(a));
    }
    void build() { // 單位矩陣
        for (int i = 0; i < r; ++i) a[i][i] = 1;
    }
};
Mat operator*(Mat x, Mat y) {
    Mat z(x.r, y.c);
    for (int i = 0; i < x.r; ++i)
        for (int j = 0; j < x.c; ++j)
            for (int k = 0; k < y.c; ++k)
                (z.a[i][j] += x.a[i][k] * y.a[k][j] % MOD) %= MOD;
    return z;
}
```

3.7 FFT

使用前要先跑過 pre_fft(); n 必須是 2^k。

```
const int MAXN = 262144 << 1; // (must be 2^k)
// before any usage, run pre_fft() first
// typedef long double ld;
typedef complex<ld> cplx; // real(), imag()
const ld PI = acos(-1);
const cplx I(0, 1);
cplx omega[MAXN + 1];
void pre_fft() {
    for (int i = 0; i <= MAXN; i++)
        omega[i] = exp(i * 2 * PI / MAXN * I);
}
void fft(int n, cplx a[], bool inv = false) {
    int basic = MAXN / n;
    int theta = basic;
    for (int m = n; m >= 2; m >>= 1) {
        int mh = m >> 1;
        for (int i = 0; i < mh; i++) {
            cplx w = omega[inv ? MAXN - (i * theta % MAXN)
                               : i * theta % MAXN];
            for (int j = i; j < n; j += m) {
                int k = j + mh;
                cplx x = a[j] - a[k];
                a[j] += a[k];
                a[k] = w * x;
            }
        }
        theta = (theta * 2) % MAXN;
    }
    int i = 0;
    for (int j = 1; j < n - 1; j++) {
        for (int k = n >> 1; k > (i ^= k); k >>= 1) {}
        if (j < i) swap(a[i], a[j]);
    }
    if (inv)
        for (i = 0; i < n; i++) a[i] /= n;
}
cplx arr[MAXN + 1];
inline void mul(int _n, ll a[], int _m, ll b[],
                ll ans[]) {
    int n = 1, sum = _n + _m - 1;
    while (n < sum) n <<= 1;
    for (int i = 0; i < n; i++) {
        double x = (i < _n ? a[i] : 0),
            y = (i < _m ? b[i] : 0);
        arr[i] = complex<double>(x + y, x - y);
    }
    fft(n, arr);
    for (int i = 0; i < n; i++) arr[i] = arr[i] * arr[i];
    fft(n, arr, true);
    for (int i = 0; i < sum; i++)
        ans[i] = (long long int)(arr[i].real() / 4 + 0.5);
}
```

3.8 質數表

```
// notPrime, 最大質因數, 質因數數量
int notPrime[MXN], fac[MXN], num[MXN];
void getPrimes() {
    notPrime[1] = 1, fac[1] = num[1];
    for (int i = 2; i < MXN; ++i) {
        if (notPrime[i]) continue;
```

```

    for (int j = i; j < MXN; j += i) {
        if (i != j) notPrime[j] = 1;
        fac[j] = i, num[j]++;
    }
}

vector<int> ret; // 質因數分解
void div(int x) {
    for (; x > 1; x /= fac[x]) ret.push_back(fac[x]);
}

```

3.9 歐拉函數

$\varphi(x)$ = 所有小於等於 x 且和 x 互質的數的數量

```

int _phi(int n) { //  $O(\sqrt{n})$ 
    int res = n, a = n;
    for (int i = 2; i * i <= a; i++) {
        if (a % i == 0) {
            res = res / i * (i - 1);
            while (a % i == 0) a /= i;
        }
    }
    if (a > 1) res = res / a * (a - 1);
    return res;
}

```

```

int phi[MXN]; // 建表 最大  $1e7$ 
void phi_table(int n) {
    phi[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (phi[i]) continue;
        for (int j = i; j <= n; j += i) {
            if (phi[j] == 0) phi[j] = j;
            phi[j] = phi[j] / i * (i - 1);
        }
    }
}

```

3.10 Miller-Rabin Algorithm

$$\text{Magic} = \begin{cases} \{2, 7, 61\}, & n < 4, 759, 123, 141 \\ \{2, 13, 23, 1662803\}, & n < 1, 122, 004, 669, 633 \\ \{2, 3, 5, 7, 11, 13\}, & n < 3, 474, 749, 660, 383 \\ \{2, 325, 9375, 28178, \\ 450775, 9780504, 1795265022\} & n < 2^{64} \end{cases}$$

```

ll mul(ll x, ll y, ll mod) {
    ll ret = x * y - (ll)((long double)x / mod * y) * mod;
    return ret < 0 ? ret + mod : ret;
    // return x * y % mod; // for __int128
}

ll magic[] = {2, 7, 61}; // 這邊填入要用於測試的數列
ll S = 3; // 測試的數列數字的數量
bool witness(ll a, ll n, ll u, int t) {
    if (!a) return 0;
    ll x = power(a, u, n);
    for (int i = 0; i < t; i++) {
        ll nx = mul(x, x, n);
        if (nx == 1 && x != 1 && x != n - 1) return 1;
        x = nx;
    }
    return x != 1;
}

bool miller_rabin(ll n) {
    int s = S; // magic number size
    // iterate s times of witness on n
    if (n < 2) return false;
    if (!(n & 1)) return (n == 2);

    // 將  $n - 1$  寫成  $u * (2^t)$  的形式
    ll u = n - 1;
    int t = 0;
    while (!(u & 1)) u >>= 1, t++;

    while (s--) {
        ll a = magic[s] % n;
        if (witness(a, n, u, t)) return false;
    }
    return true;
}

```

3.11 Pollard's Rho Algorithm

```

vector<ll> ret;
ll f(ll x, ll c, ll mod) {
    return (mul(x, x, mod) + c) % mod;
}

ll pollard_rho(ll n) {
    ll c = 1, x = 0, y = 0, p = 2, q, t = 0;
    while (t++ % 128 || gcd(p, n) == 1) {
        if (x == y) c++, y = f(x = 2, c, n);
        if (q = mul(p, abs(x - y), n)) p = q;
        x = f(x, c, n);
        y = f(f(y, c, n), c, n);
    }
    return gcd(p, n);
}

void fact(ll x) { // 透過遞迴找質數
    if (miller_rabin(x)) {
        ret.push_back(x);
        return;
    }
    ll f = pollard_rho(x);
    fact(f);
    fact(x / f);
}

```

3.12 高斯消去法

```

/**
 * mat[n][m] 為  $n$  個  $m$  元一次連例方程式的增廣矩陣
 * ans[m] 輸出  $x_m$  的解
 * led[n] 紀錄第  $n$  列的領導係數的位置
 * 若無解則回傳 false
 */
ll mat[MXN][MXN], ans[MXN], led[MXN];
bool gaussian(int n, int m) {
    for (int i = 0; i < n; i++) { // 高斯消元法
        // 第  $i$  列的領導項
        while (led[i] <= m && mat[i][led[i]] == 0) led[i]++;
        // 當每項係數皆為 0 時,  $mat_{im} == b_i != 0 \Rightarrow$  無解
        if (led[i] == m && mat[i][m] != 0) return false;
        int c = led[i];
        for (int j = 0; j < n; j++) {
            // 消掉第  $j$  列的這項係數, 使  $mat_{jc} = 0$ 
            if (i == j) continue;
            int r = mat[j][c] / mat[i][c];
            //  $mat_j -= mat_i * (mat_{jc} / mat_{ic})$ 
            for (int k = c; k <= m; k++)
                mat[j][k] = mat[j][k] - mat[i][k] * r;
        }
    }
    for (int i = 0; i < m; i++) {
        // 紀錄  $x_c$  的解為  $b_i / mat_{ic}$ 
        ans[led[i]] = mat[i][m] / mat[i][led[i]];
    }
    return true; // 有解
}

```

3.13 約瑟夫問題

```

int josephus(int n, int m) { //  $n$  人每  $m$  次
    int ans = 0;
    for (int i = 1; i <= n; ++i) ans = (ans + m) % i;
    return ans;
}

```

3.14 莫比烏斯函數

```

vector<int> pri;
bool not_prime[MXN];
int mu[MXN];

void pre(int n) {
    mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (!not_prime[i]) {
            mu[i] = -1;
            pri.push_back(i);
        }
        for (int pri_j : pri) {
            if (i * pri_j > n) break;
            not_prime[i * pri_j] = true;
            if (i % pri_j == 0) {

```

```

        mu[i * pri_j] = 0;
        break;
    }
    mu[i * pri_j] = -mu[i];
}
}
}

```

4 Graph

4.1 Topological Sort

```

vector<int> edge[MAXN], ans;
int deg[MAXN]; // in degree
void topo(int n) { // 1-base
    queue<int> que;
    for (int i = 1; i <= n; i++)
        if (!deg[i]) que.push(i);
    while (!que.empty()) {
        int u = que.front();
        que.pop();
        ans.push_back(u);
        // 結果存 ans
        for (int v : edge[u]) {
            deg[v]--;
            if (!deg[v]) que.push(v);
        }
    }
}

```

4.2 Tarjan's Algorithm for BCC

```

struct BccVertex {
    int n, nScc, step, dfn[MXN], low[MXN];
    vector<int> E[MXN], sccv[MXN];
    int top, stk[MXN];
    void init(int _n) {
        n = _n;
        nScc = step = 0;
        for (int i = 0; i < n; i++)
            E[i].clear();
    }
    void addEdge(int u, int v) {
        E[u].PB(v);
        E[v].PB(u);
    }
    void DFS(int u, int f) {
        dfn[u] = low[u] = step++;
        stk[top++] = u;
        for (auto v : E[u]) {
            if (v == f) continue;
            if (dfn[v] == -1) {
                DFS(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u]) {
                    int z;
                    sccv[nScc].clear();
                    do {
                        z = stk[--top];
                        sccv[nScc].PB(z);
                    } while (z != v);
                    sccv[nScc++].PB(u);
                }
            } else low[u] = min(low[u], dfn[v]);
        }
    }
    vector<vector<int>> solve() {
        vector<vector<int>> res;
        for (int i = 0; i < n; i++)
            dfn[i] = low[i] = -1;
        for (int i = 0; i < n; i++)
            if (dfn[i] == -1) {
                top = 0;
                DFS(i, i);
            }
        REP(i, nScc) res.PB(sccv[i]);
        return res;
    }
} graph;

```

4.3 Bellmen-Ford Algorithm

```

struct Edge {
    int from, to, weight;

```

```

} edge[MAXN];
int dis[MAXN];
void bellmen_ford(int n, int m) {
    dis[1] = 0; // 起點的距離一定是 0
    for (int j = 0; j < n - 1;
        j++) { // n 個節點要跑 n - 1 次
        for (int i = 0; i < m; i++) { // 對於所有邊都嘗試鬆弛
            if (dis[edge[i].to] >
                dis[edge[i].from] + edge[i].weight)
                dis[edge[i].to] =
                    dis[edge[i].from] + edge[i].weight;
        }
    }
}

```

4.4 Dijkstra's Algorithm

```

vector<pii> vec[N]; // vec[u] = {v, w}: u 為起點 v 為終點
// w 為路權
bool vis[N]; // 紀錄該節點是否走過
vector<int> dijkstra(int s) { // 起點
    vector<int> dis(N);
    for (int i = 0; i < N; i++) // 初始化
        dis[i] =
            INF; // 值要設為比可能的最短路徑權重還要大的值
    dis[s] = 0;
    priority_queue<pii, vector<pii>, greater<pii>>
        pq; // 以小到排序
    // {w, v}: w 為路權 v 為節點
    pq.push({dis[s], s});
    while (pq.empty() == 0) {
        int u = pq.top().second;
        pq.pop();
        if (vis[u]) // 走過的不要再走
            continue;
        vis[u] = 1;
        for (auto i : vec[u]) {
            int v = i.first, w = i.second;
            if (dis[u] + w < dis[v]) { // 鬆弛
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }
    return dis;
}

```

4.5 Dinic's Algorithm for Maximum Flow

```

#define SZ(x) ((int)x.size())
#define PB push_back
struct Dinic {
    struct Edge {
        int v, f, re;
    };
    int n, s, t, level[MXN];
    vector<Edge> E[MXN];
    void init(int _n, int _s, int _t) {
        n = _n;
        s = _s;
        t = _t;
        for (int i = 0; i < n; i++) E[i].clear();
    }
    void add_edge(int u, int v, int f) {
        E[u].PB({v, f, SZ(E[v])});
        E[v].PB({u, 0, SZ(E[u]) - 1});
    }
    bool BFS() {
        for (int i = 0; i < n; i++) level[i] = -1;
        queue<int> que;
        que.push(s);
        level[s] = 0;
        while (!que.empty()) {
            int u = que.front();
            que.pop();
            for (auto it : E[u]) {
                if (it.f > 0 && level[it.v] == -1) {
                    level[it.v] = level[u] + 1;
                    que.push(it.v);
                }
            }
        }
    }
    return level[t] != -1;
}

```

```

}
int DFS(int u, int nf) {
    if (u == t) return nf;
    int res = 0;
    for (auto &it : E[u]) {
        if (it.f > 0 && level[it.v] == level[u] + 1) {
            int tf = DFS(it.v, min(nf, it.f));
            res += tf;
            nf -= tf;
            it.f -= tf;
            E[it.v][it.re].f += tf;
            if (nf == 0) return res;
        }
    }
    if (!res) level[u] = -1;
    return res;
}
int flow(int res = 0) {
    while (BFS()) res += DFS(s, 2147483647);
    return res;
}
} flow;
/*
    查找二分圖配對點:
    在 add_edge 時寫上 index[u].push_back({v,
    flow.E[u].size()});

    在 flow 完之後寫上這個
    for (int i = 1; i <= n; i++) { // 遍歷所有節點
        for (pair<int, int> j : index[i]) {
            if (flow.E[i][j.second - 1].f == 0)
                cout << i << ' ' << j.first << '\n';
        }
    }
*/

```

4.6 Dominator Tree

```

#define REP(i, s, e) for (int i = (s); i <= (e); i++)
#define REPD(i, s, e) for (int i = (s); i >= (e); i--)
struct DominatorTree { // O(N)
    int n, s;
    vector<int> g[MAXN], pred[MAXN];
    vector<int> cov[MAXN];
    int dfn[MAXN], nfd[MAXN], ts;
    int par[MAXN]; // idom[u] s到u的最後一個必經點
    int sdom[MAXN], idom[MAXN];
    int mom[MAXN], mn[MAXN];
    inline bool cmp(int u, int v) {
        return dfn[u] < dfn[v];
    }
    int eval(int u) {
        if (mom[u] == u) return u;
        int res = eval(mom[u]);
        if (cmp(sdom[mn[mom[u]]], sdom[mn[u]]))
            mn[u] = mn[mom[u]];
        return mom[u] = res;
    }
    void init(int _n, int _s) {
        ts = 0;
        n = _n;
        s = _s;
        REP(i, 1, n) g[i].clear(), pred[i].clear();
    }
    void addEdge(int u, int v) {
        g[u].push_back(v);
        pred[v].push_back(u);
    }
    void dfs(int u) {
        ts++;
        dfn[u] = ts;
        nfd[ts] = u;
        for (int v : g[u])
            if (dfn[v] == 0) {
                par[v] = u;
                dfs(v);
            }
    }
    void build() {
        REP(i, 1, n) {
            dfn[i] = nfd[i] = 0;
            cov[i].clear();
            mom[i] = mn[i] = sdom[i] = i;

```

```

        }
        dfs(s);
        REPD(i, n, 2) {
            int u = nfd[i];
            if (u == 0) continue;
            for (int v : pred[u])
                if (dfn[v]) {
                    eval(v);
                    if (cmp(sdom[mn[v]], sdom[u]))
                        sdom[u] = sdom[mn[v]];
                }
            cov[sdom[u]].push_back(u);
            mom[u] = par[u];
            for (int w : cov[par[u]]) {
                eval(w);
                if (cmp(sdom[mn[w]], par[u])) idom[w] = mn[w];
                else idom[w] = par[u];
            }
            cov[par[u]].clear();
        }
        REP(i, 2, n) {
            int u = nfd[i];
            if (u == 0) continue;
            if (idom[u] != sdom[u]) idom[u] = idom[idom[u]];
        }
    }
} domT;

```

4.7 Floyd-Warshall Algorithm

```

int dis[N][N];
void init(int n) {
    for (int i = 0; i <= n; i++) { // 初始化
        for (int j = 0; j <= n; j++)
            dis[i][j] = INF;
        dis[i][i] = 0;
    }
}
void floyd_warshall(int n) {
    for (int k = 0; k < n; k++) { // 窮舉中繼點 k
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) { // 窮舉點對 (i, j)
                dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
            }
        }
    }
}

```

4.8 Maximum Clique

```

#define N 111 // 80 ~ 100 左右
struct MaxClique { // 0-base
    typedef bitset<N> Int;
    Int linkto[N], v[N];
    int n;
    void init(int _n) {
        n = _n;
        for (int i = 0; i < n; i++) {
            linkto[i].reset();
            v[i].reset();
        }
    }
    void addEdge(int a, int b) { v[a][b] = v[b][a] = 1; }
    int popcount(const Int& val) { return val.count(); }
    int lowbit(const Int& val) { return val._Find_first(); }
    int ans, stk[N];
    int id[N], di[N], deg[N];
    Int cans;
    void maxclique(int elem_num, Int candi) {
        if (elem_num > ans) {
            ans = elem_num;
            cans.reset();
            for (int i = 0; i < elem_num; i++)
                cans[id[stk[i]]] = 1;
        }
        int potential = elem_num + popcount(candi);
        if (potential <= ans) return;
        int pivot = lowbit(candi);
        Int smaller_candi = candi & (~linkto[pivot]);
        while (smaller_candi.count() && potential > ans) {
            int next = lowbit(smaller_candi);
            candi[next] = !candi[next];
            smaller_candi[next] = !smaller_candi[next];
            potential--;

```

```

        if (next == pivot ||
            (smaller_candi & linkto[next]).count()) {
            stk[elem_num] = next;
            maxclique(elem_num + 1, candi & linkto[next]);
        }
    }
}

int solve() {
    for (int i = 0; i < n; i++) {
        id[i] = i;
        deg[i] = v[i].count();
    }
    sort(id, id + n, [&](int id1, int id2) {
        return deg[id1] > deg[id2];
    });
    for (int i = 0; i < n; i++)
        di[id[i]] = i;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (v[i][j]) linkto[di[i]][di[j]] = 1;
    Int cand;
    cand.reset();
    for (int i = 0; i < n; i++)
        cand[i] = 1;
    ans = 1;
    cans.reset();
    cans[0] = 1;
    maxclique(0, cand);
    return ans;
}
} solver;

```

4.9 Kosaraju's Algorithm for SCC

```

struct Scc { // 新的 DAG 可能會有多重邊
#define PB push_back
    int n, nScc, vst[MXN], bln[MXN];
    vector<int> E[MXN], rE[MXN], vec, dag[MXN];
    set<int> vis[MXN];
    void init(int _n) {
        n = _n;
        for (int i = 0; i <= n; i++) {
            E[i].clear(), rE[i].clear();
            dag[i].clear(), vis[i].clear();
            bln[i] = -1;
        }
    }
    void addEdge(int u, int v) {
        E[u].PB(v);
        rE[v].PB(u);
    }
    void DFS(int u) {
        vst[u] = 1;
        for (auto v : E[u])
            if (!vst[v]) DFS(v);
        vec.PB(u);
    }
    void rDFS(int u) {
        vst[u] = 1;
        bln[u] = nScc; // 編號是 0-base
        for (auto v : rE[u])
            if (!vst[v]) rDFS(v);
    }
    void build_new_dag() {
        for (int i = 0; i < n; i++) {
            for (int &j : E[i]) {
                if (bln[i] != bln[j] &&
                    vis[bln[i]].count(bln[j]) == 0) {
                    dag[bln[i]].push_back(bln[j]);
                    vis[bln[i]].insert(bln[j]);
                }
            }
        }
    }
    void solve() {
        nScc = 0;
        vec.clear();
        fill(vst, vst + n + 1, 0);
        for (int i = 0; i < n; i++)
            if (!vst[i]) DFS(i);
        reverse(vec.begin(), vec.end());
        fill(vst, vst + n + 1, 0);
        for (auto v : vec) {
            if (!vst[v]) {

```

```

                rDFS(v);
                nScc++;
            }
        }
        build_new_dag();
    }
};

```

4.10 Kruskal's Algorithm for MST

```

void kruskal() {
    sort(graph, graph + m); // 將邊照大小排序
    int ans = 0;

    for (int i = 0; i < m; i++) { //>:)
        if (find(graph[i].u) != find(graph[i].v)) {
            // 如果兩點未聯通
            merge(graph[i].u, graph[i].v);
            // 將兩點設成同一個集合
            ans += graph[i].w; // 權重加進答案
            if (sz[find(graph[i].u)] == n)
                break; // 當並查集大小等價於樹內點的數量
        }
    }
}

```

4.11 Kuhn-Munkre Algorithm

```

struct KM { // max weight, for min negate the weights
    int n, mx[MXN], my[MXN], pa[MXN];
    ll g[MXN][MXN], lx[MXN], ly[MXN], sy[MXN];
    bool vx[MXN], vy[MXN];
    void init(int _n) { // 1-based
        n = _n; // 初始化
        for (int i = 1; i <= n; i++)
            fill(g[i], g[i] + n + 1, 0);
    }
    void addEdge(int x, int y, ll w) { // 加邊
        g[x][y] = w;
    }
    void augment(int y) {
        for (int x, z; y; y = z)
            x = pa[y], z = mx[x], my[y] = x, mx[x] = y;
    }
    void bfs(int st) {
        for (int i = 1; i <= n; ++i)
            sy[i] = INF, vx[i] = vy[i] = 0;
        queue<int> q;
        q.push(st);
        for (;;) {
            while (q.size()) {
                int x = q.front();
                q.pop();
                vx[x] = 1;
                for (int y = 1; y <= n; ++y)
                    if (!vy[y]) {
                        ll t = lx[x] + ly[y] - g[x][y];
                        if (t == 0) {
                            pa[y] = x;
                            if (!my[y]) {
                                augment(y);
                                return;
                            }
                        }
                        vy[y] = 1, q.push(my[y]);
                    } else if (sy[y] > t) pa[y] = x, sy[y] = t;
            }
            ll cut = INF;
            for (int y = 1; y <= n; ++y)
                if (!vy[y] && cut > sy[y]) cut = sy[y];
            for (int j = 1; j <= n; ++j) {
                if (vx[j]) lx[j] -= cut;
                if (vy[j]) ly[j] += cut;
                else sy[j] -= cut;
            }
            for (int y = 1; y <= n; ++y)
                if (!vy[y] && sy[y] == 0) {
                    if (!my[y]) {
                        augment(y);
                        return;
                    }
                }
            vy[y] = 1, q.push(my[y]);
        }
    }
}

```

```

    }
}
ll solve() { // 解決
    fill(mx, mx + n + 1, 0);
    fill(my, my + n + 1, 0);
    fill(lx, lx + n + 1, 0);
    fill(lx, lx + n + 1, -INF);
    for (int x = 1; x <= n; ++x)
        for (int y = 1; y <= n; ++y)
            lx[x] = max(lx[x], g[x][y]);
    for (int x = 1; x <= n; ++x)
        bfs(x);
    ll ans = 0;
    for (int y = 1; y <= n; ++y)
        ans += g[my[y]][y]; // 答案存在 my[]
    return ans;
}
} graph; // 找二分圖最大權完美匹配

```

4.12 Max-flow-min-cost

```

struct zkwflow {
    static const int maxN = 100000;
    struct Edge {
        int v, f, re;
        ll w;
    };
    int n, s, t, ptr[maxN];
    bool vis[maxN];
    ll dis[maxN];
    vector<Edge> E[maxN];
    void init(int _n, int _s, int _t) {
        n = _n, s = _s, t = _t;
        for (int i = 0; i < n; i++)
            E[i].clear();
    }
    void add_edge(int u, int v, int f, ll w) {
        E[u].push_back({v, f, (int)E[v].size(), w});
        E[v].push_back({u, 0, (int)E[u].size() - 1, -w});
    }
    bool SPFA() {
        fill_n(dis, n, LLONG_MAX);
        fill_n(vis, n, false);
        queue<int> q;
        q.push(s);
        dis[s] = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            vis[u] = false;
            for (auto &it : E[u]) {
                if (it.f > 0 && dis[it.v] > dis[u] + it.w) {
                    dis[it.v] = dis[u] + it.w;
                    if (!vis[it.v]) {
                        vis[it.v] = true;
                        q.push(it.v);
                    }
                }
            }
        }
        return dis[t] != LLONG_MAX;
    }
    int DFS(int u, int nf) {
        if (u == t) return nf;
        int res = 0;
        vis[u] = true;
        for (int &i = ptr[u]; i < (int)E[u].size(); i++) {
            auto &it = E[u][i];
            if (it.f > 0 && dis[it.v] == dis[u] + it.w && !vis[it.v]) {
                int tf = DFS(it.v, min(nf, it.f));
                res += tf, nf -= tf, it.f -= tf;
                E[it.v][it.re].f += tf;
                if (nf == 0) {
                    vis[u] = false;
                    break;
                }
            }
        }
        return res;
    }
    pair<int, ll> flow() {
        int flow = 0;
        ll cost = 0;

```

```

        while (SPFA()) {
            fill_n(ptr, n, 0);
            int f = DFS(s, INT_MAX);
            flow += f;
            cost += dis[t] * f;
        }
        return {flow, cost};
    } // reset: do nothing
} flow;

```

4.13 Kuhn Algorithm for Matching

```

vector<int> g[MXN];
bool vis[MXN];
int S[MXN]; // S 為紀錄這個點與誰匹配
int n, ans; // n: 左集合數量, ans: 紀錄答案

bool dfs(int u) { // 找最大匹配
    for (int &v : g[u]) {
        if (!vis[v]) {
            vis[v] = true;
            if (S[v] == -1 || dfs(S[v])) {
                S[v] = u;
                return true;
            }
        }
    }
    return false;
}

void solve() { // 記得每次使用需清空 vis
    memset(S, -1, sizeof(S));
    ans = 0;
    for (int i = 1; i <= n; i++) { // 跑左集合
        memset(vis, 0, sizeof(vis));
        if (dfs(i)) ans++;
    }
}

```

5 Tree Algorithms

5.1 時間戳記 & 倍增表 (初始化)

```

const int LG_MXN = __lg(MXN) + 2;
vector<int> edge[MXN]; // i 有哪些小孩
int timing = 1, lgN, n;
int in[MXN], out[MXN], anc[MXN][LG_MXN], depth[MXN];
void dfs_init(int u, int f) { // 0 倍祖先、時間戳記
    in[u] = timing++; // 這時進入 u
    anc[u][0] = f; // now 的 0 倍祖先是 f
    for (int v : edge[u]) {
        if (v == f) continue;
        depth[v] = depth[u] + 1;
        dfs_init(v, u);
    }
    out[u] = timing++; // 這時離開 u
}

void build_table(int n) { // 初始化
    lgN = __lg(n) + 1;
    dfs_init(1, 0);
    for (int i = 1; i <= lgN; i++) // 建立倍增表
        for (int now = 1; now <= n; now++)
            anc[now][i] = anc[anc[now][i - 1]][i - 1];
}

```

5.2 LCA

```

bool is_ancestor(int u, int v) { // 判斷 u 是否 v 是祖先
    return (in[u] < in[v] && out[v] < out[u]);
}

int getLCA(int x, int y) { // 找最近共同祖先
    if (is_ancestor(x, y)) // 如果 x 為 y 的祖先
        return x; // 則 LCA 為 y
    if (is_ancestor(y, x)) // 如果 y 為 x 的祖先
        return y; // 則 LCA 為 x
    for (int i = lgN; i >= 0; i--) // 逼近 LCA
        if (!is_ancestor(anc[x][i], y) && anc[x][i])
            x = anc[x][i];
    return anc[x][0]; // 回傳此點的父節點即為答案
}

```


5.3 Euler Tour for LCA

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 5;
vector<int> tour, edge[N];
int timing = 0;
int in[N], out[N];
void dfs(int u, int fa) {
    tour.push_back(u); // 進入的時間點
    in[u] = timing++;
    for (auto v : edge[u]) {
        if (v == fa) continue;
        dfs(v, u);
        tour.push_back(u); // 走完其中一個孩子 再走回自己
    }
    out[u] = timing++;
}
void build(vector<int>& tour) { // 把tour丟進線段樹初始化
    seg.build(tour); // 除了tour, 還要建每個點的深度
}
int query(int x, int y) {
    if (is_ancestor(x, y)) return x; // LCA(x,y) = x
    if (is_ancestor(y, x)) return y; // LCA(x,y) = y;
    // 如果「y的區間」在「x的區間」的左邊, 就交換
    if (out[y] < in[x]) swap(x, y);
    // 取得區間內哪個位置的深度是最低的
    int index = seg.query(out[x], in[y]);
    return tour[index];
}
```

5.4 樹上差分

```
int tag[N], f[N], ans[N], root;
void add(int x, int y) {
    int lca = getLCA(x, y);
    tag[x]++;
    tag[y]++;
    tag[lca]--;
    if (lca != root) tag[f[lca]]--;
}
int dfs_add(int now, int fa) {
    int diff = tag[now];
    for (auto nxt : edge[now]) {
        if (nxt == fa) continue;
        diff += dfs_add(nxt, now);
    }
    return ans[now] = diff;
}
```

5.5 DSU on Tree

```
int find(int x) {
    if (f[x] == x) return x;
    else return f[x] = find(f[x]);
}
void merge(int x, int y) {
    x = find(x), y = find(y);
    if (sz[x] < sz[y]) swap(x, y);
    sz[x] += sz[y];
    f[y] = x;
}
void init() {
    // 初始化一開始大小都為1 (每群一開始都是獨立一個)
    for (int i = 1; i <= n; i++) sz[i] = 1;
    for (int i = 1; i <= n; i++) f[i] = i;
}
```

5.6 樹鏈剖分

```
const int MXN = 2e5 + 7;
int top[MXN], son[MXN], dfn[MXN], rnk[MXN], dep[MXN],
    father[MXN];
vector<int> edge[MXN];
int dfs1(int v, int fa, int d) {
    int maxsz = -1, maxu, total = 1;
    dep[v] = d;
    father[v] = fa;
    for (int u : edge[v]) {
        if (fa == u) continue;
        int temp = dfs1(u, v, d + 1);
        total += temp;
        if (temp > maxsz) {
```

```
            maxsz = temp;
            maxu = u;
        }
    }
    if (maxsz == -1) son[v] = -1;
    else son[v] = maxu;
    return total;
}
int times = 1;
void dfs2(int v, int fa) {
    rnk[times] = v;
    dfn[v] = times++;
    top[v] = (fa == -1 || son[fa] != v ? v : top[fa]);
    if (son[v] != -1) dfs2(son[v], v);
    for (int u : edge[v]) {
        if (fa == u || u == son[v]) continue;
        dfs2(u, v);
    }
}
// rnk: 剖分後的編號 (rnk[時間] = 原點)
// dfn: 剖分後的編號 (dfn[原點] = 時間)
// top: 剖分的頭頭 son: 剖分的重兒子
```

6 String Algorithms

6.1 Suffix Array

```
const int MXN = 100005;
struct SA {
    int n;
    vector<int> rk, oldrk, sa, h;
    string s;
    SA(string& t)
        : s(t),
          n(t.size()),
          rk(n << 1),
          oldrk(n << 1),
          sa(n),
          h(n) {
        for (int i = 0; i < n; i++)
            rk[i] = s[i] - 'a' + 1, sa[i] = i;
        for (int w = 1; w < n; w <= 1) {
            sort(sa.begin(), sa.end(), [&](int x, int y) {
                return (rk[x] == rk[y]) ? (rk[x + w] < rk[y + w])
                    : (rk[x] < rk[y]);
            });
            oldrk = rk;
            rk[sa[0]] = 1;
            for (int i = 1, p = 1; i < n; i++) {
                if (oldrk[sa[i]] == oldrk[sa[i - 1]] &&
                    oldrk[sa[i] + w] == oldrk[sa[i - 1] + w])
                    rk[sa[i]] = p;
                else rk[sa[i]] = ++p;
            }
        }
        for (int i = 0, k = 0; i < n; i++) {
            if (rk[i] == 1) continue;
            if (k) k--;
            while (s[i + k] == s[sa[rk[i] - 2] + k]) k++;
            h[rk[i] - 1] = k;
        }
    }
    void find(string p) { // binary search
        int l = 0, r = n, m = p.size();
        if (m > n) return;
        for (int i = 0; i < m; i++) {
            auto cmp = [&](int lhs, int rhs) {
                return s[lhs + i] < rhs;
            };
            auto nl =
                lower_bound(sa + 1, sa + r, p[i], cmp) - sa;
            auto nr =
                lower_bound(sa + 1, sa + r, p[i] + 1, cmp) - sa;
            l = nl, r = nr;
            if (l >= r) break;
        }
        cout << (r > 1 ? "YES" : "NO") << "\n";
    }
};
```

6.2 Suffix Array (SAIS)

// 此為後綴陣列模板

```
// 用法:
// 把要處理的字串轉成整數陣列傳入suffix_array(), 會得到SA
// 與H維結果
const int N = 500010; // 有可能要在這就寫原本字串的兩倍長
struct SA {
#define REP(i, n) for (int i = 0; i < int(n); i++)
#define REP1(i, a, b) for (int i = (a); i <= int(b); i++)
    bool _t[N * 2];
    int _s[N * 2], _sa[N * 2], _c[N * 2], x[N], _p[N],
        _q[N * 2], hei[N], r[N];
    int operator[](int i) { return _sa[i]; }
    void build(int* s, int n, int m) {
        memcpy(_s, s, sizeof(int) * n);
        sais(_s, _sa, _p, _q, _t, _c, n, m);
        mkhei(n);
    }
    void mkhei(int n) {
        REP(i, n) r[_sa[i]] = i;
        hei[0] = 0;
        REP(i, n) if (r[i]) {
            int ans = i > 0 ? max(hei[r[i] - 1] - 1, 0) : 0;
            while (_s[i + ans] == _s[_sa[r[i] - 1] + ans])
                ans++;
            hei[r[i]] = ans;
        }
    }
    void sais(int* s, int* sa, int* p, int* q, bool* t,
        int* c, int n, int z) {
        bool uniq = t[n - 1] = true, neq;
        int nn = 0, nmzx = -1, *nsa = sa + n, *ns = s + n,
            lst = -1;
#define MS0(x, n) memset((x), 0, n * sizeof(*(x)))
#define MAGIC(XD) \
        MS0(sa, n); \
        memcpy(x, c, sizeof(int) * z); \
        XD; \
        memcpy(x + 1, c, sizeof(int) * (z - 1)); \
        REP(i, n) \
        if (sa[i] && !t[sa[i] - 1]) \
            sa[x[s[sa[i] - 1]]++] = sa[i] - 1; \
        memcpy(x, c, sizeof(int) * z); \
        for (int i = n - 1; i >= 0; i--) \
            if (sa[i] && t[sa[i] - 1]) \
                sa[--x[s[sa[i] - 1]]] = sa[i] - 1; \
        MS0(c, z); \
        REP(i, n) uniq &= ++c[s[i]] < 2; \
        REP(i, z - 1) c[i + 1] += c[i]; \
        if (uniq) { \
            REP(i, n) sa[--c[s[i]]] = i; \
            return; \
        } \
        for (int i = n - 2; i >= 0; i--) \
            t[i] = \
                (s[i] == s[i + 1] ? t[i + 1] : s[i] < s[i + 1]); \
        MAGIC(REP1(i, 1, n - 1) if (t[i] && !t[i - 1]) \
            sa[--x[s[i]]] = p[q[i] = nn++] = i); \
        REP(i, n) if (sa[i] && t[sa[i]] && !t[sa[i] - 1]) { \
            neq = lst < 0 || memcmp(s + sa[i], s + lst, \
                (p[q[sa[i]] + 1] - sa[i]) * \
                sizeof(int)); \
            ns[q[lst = sa[i]]] = nmzx += neq; \
        } \
        sais(ns, nsa, p + nn, q + n, t + n, c + z, nn, \
            nmzx + 1); \
        MAGIC(for (int i = nn - 1; i >= 0; i--) \
            sa[--x[s[p[nsa[i]]]]] = p[nsa[i]]); \
    }
} sa;
int H[N], SA[N]; // SA: 後綴陣列的Length
// H[i]: SA[i]與SA[i-1]的共同子字串長
void suffix_array(int* ip, int len) {
    // should padding a zero in the back
    // ip is int array, len is array length
    // ip[0..n-1] != 0, and ip[len] = 0
    ip[len++] = 0;
    sa.build(ip, len, 128);
    for (int i = 0; i < len; i++) {
        H[i] = sa.hei[i + 1];
        SA[i] = sa._sa[i + 1];
    }
    // resulting height, sa array \in [0, len)
}
```

6.3 AC Automata

```
struct ACautomata {
    struct Node {
        int cnt, i;
        Node *go[26], *fail, *dic;
        Node() {
            cnt = 0;
            fail = 0;
            dic = 0;
            i = 0;
            memset(go, 0, sizeof(go));
        }
    } pool[1048576], *root;
    int nMem, n_pattern;
    Node* new_Node() {
        pool[nMem] = Node();
        return &pool[nMem++];
    }
    void init() {
        nMem = 0;
        root = new_Node();
        n_pattern = 0;
        add("");
    }
    void add(const string& str) { insert(root, str, 0); }
    void insert(Node* cur, const string& str, int pos) {
        for (int i = pos; i < str.size(); i++) {
            if (!cur->go[str[i] - 'a'])
                cur->go[str[i] - 'a'] = new_Node();
            cur = cur->go[str[i] - 'a'];
        }
        cur->cnt++;
        cur->i = n_pattern++;
    }
    void make_fail() {
        queue<Node*> que;
        que.push(root);
        while (!que.empty()) {
            Node* fr = que.front();
            que.pop();
            for (int i = 0; i < 26; i++) {
                if (fr->go[i]) {
                    Node* ptr = fr->fail;
                    while (ptr && !ptr->go[i]) ptr = ptr->fail;
                    fr->go[i]->fail = ptr = (ptr ? ptr->go[i] : root);
                    fr->go[i]->dic = (ptr->cnt ? ptr : ptr->dic);
                    que.push(fr->go[i]);
                }
            }
        }
    }
    void query(string s) {
        Node* cur = root;
        for (int i = 0; i < (int)s.size(); i++) {
            while (cur && !cur->go[s[i] - 'a']) cur = cur->fail;
            cur = (cur ? cur->go[s[i] - 'a'] : root);
            if (cur->i >= 0) ans[cur->i]++;
            for (Node* tmp = cur->dic; tmp; tmp = tmp->dic)
                ans[tmp->i]++;
        }
    }
    // ans[i] : number of occurrence of pattern i
} AC;
```

6.4 Suffix Automata

```
struct SAM {
    struct State {
        int len;
        State* link;
        map<char, State*> next;
    };
    State *first, *last;
    SAM() { first = last = new State(); }
    SAM(string& s) : SAM() {
        for (int i = 0; i < s.size(); i++) add(s[i]);
    }
    void add(char ch) {
        State* nw = new State();
        nw->len = last->len + 1;
        nw->link = first;
        State* cur = last;
        while (cur && !cur->next[ch]) {
```

```

    cur->next[ch] = nw;
    cur = cur->link;
}
if (cur && cur->next[ch] != nw) {
    State* mid = cur->next[ch];
    if (cur->len + 1 == mid->len) {
        nw->link = mid;
    } else {
        State* clone = new State(*mid);
        clone->len = cur->len + 1;
        nw->link = mid->link = clone;
        while (cur && cur->next[ch] == mid) {
            cur->next[ch] = clone;
            cur = cur->link;
        }
    }
}
last = nw;
}
};

```

6.5 Z-algorithm

```

int z[MAXN];
void Z_value(const string& s) {
    // z[i] = lcp(s[1...], s[i...])
    int i, j, left, right, len = s.size();
    left = right = 0;
    z[0] = len;
    for (i = 1; i < len; i++) {
        j = max(min(z[i - left], right - i), 0);
        for (; i + j < len && s[i + j] == s[j]; j++);
        z[i] = j;
        if (i + z[i] > right) {
            right = i + z[i];
            left = i;
        }
    }
}

```

6.6 Knuth-Morris-Pratt Algorithm

```

vector<int> KMP(string s, string t) {
    s = t + '@' + s;
    int sz = s.size();
    vector<int> pi(sz);
    for (int i = 1; i < sz; i++) {
        int len = pi[i - 1];
        while (len != 0 && s[i] != s[len]) len = pi[len - 1];
        if (s[i] == s[len]) pi[i] = len + 1;
    }
    return pi;
}

```

6.7 Manacher's Algorithm

```

vector<int> d1(n), d2(n);
for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
    while (0 <= i - k && i + k < n && s[i - k] == s[i + k])
        k++;
    d1[i] = k--;
    if (i + k > r) l = i - k, r = i + k;
}
for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
    while (0 <= i - k - 1 && i + k < n &&
           s[i - k - 1] == s[i + k])
        k++;
    d2[i] = k--;
    if (i + k > r) l = i - k - 1, r = i + k;
}

```

6.8 Minimum Rotation

```

int min_rotation(string s) {
    int a = 0, N = s.size();
    s += s;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            if (a + j == i || s[a + j] < s[i + j]) {
                i += max(0, j - 1);
                break;
            }
        }
    }
}

```

```

    if (s[a + j] > s[i + j]) {
        a = i;
        break;
    }
}
return a;
}
// rotate(begin(s), begin(s) + minRotation(s), end(s))
// 上面註解可以直接讓一個字串做 rotate，得到字典序最小的
// rotation

```

6.9 Rolling Hash

```

ll P1 = 75577, P2 = 12721, MOD = 998244353;
pii hashes[MAXN], hashes2[MAXN];
string s1, s2;
void build(string s, pii hash) {
    ll val1 = 0, val2 = 0;
    for (int i = 0; i < s.length(); i++) {
        val1 = (val1 * P1 % MOD + s[i]) % MOD;
        val2 = (val2 * P2 % MOD + s[i]) % MOD;
        hash[i] = {val1, val2};
    }
}
ll minus(ll a, ll b) { return (a - b + MOD) % MOD; }
ll cnt_occur() { // 檢查
    int n = s1.size(), k = s2.size();
    ll cnt = (hashs2[k - 1].x == hashes[k - 1].x &&
               hashes2[k - 1].y == hashes[k - 1].y);
    ll val1 = 0, val2 = 0;
    for (int i = 1, j = k; j < n; j++, i++) {
        val1 = minus(hashes[j].x,
                      hashes[i - 1].x * fpow(P1, j - i + 1));
        val2 = minus(hashes[j].y,
                      hashes[i - 1].y * fpow(P2, j - i + 1));
        if (hashs2[k - 1].x == val1 &&
            hashes2[k - 1].y == val2)
            cnt++;
    }
    return cnt;
}

```

6.10 Another Hash

```

const int M = 1e9 + 7;
const int C = 13331;
struct HashString {
    int n;
    vector<ll> pows, sums;
    HashString(string s)
        : n(s.size()), pows(n + 1, 1), sums(n + 1) {
        for (int i = 1; i <= n; i++) {
            pows[i] = pows[i - 1] * C % M;
            sums[i] = (sums[i - 1] * C + s[i - 1]) % M;
        }
    }
    ll hash(int l, int r) {
        ll h = sums[r] - sums[l] * pows[r - l];
        return (h % M + M) % M;
    }
};

```

7 Geometry

7.1 Point

```

using ld = long double;
template<class T>
struct pt {
    T x, y;
    pt(T _x, T _y) : x(_x), y(_y) {}
    pt() : x(0), y(0) {}
    pt operator * (T c) { return pt(x*c, y*c); }
    pt operator / (T c) { return pt(x/c, y/c); }
    pt operator + (pt a) { return pt(x+a.x, y+a.y); }
    pt operator - (pt a) { return pt(x-a.x, y-a.y); }
    T operator * (pt a) { return x*a.x + y*a.y; }
    T operator ^ (pt a) { return x*a.y - y*a.x; }

    auto operator<=>(pt o) const { return (x != o.x) ? x <=
        o.x : y <= o.y; } // c++20

```

```

__bool operator < (Pt a) const { return x < a.x || (x == a
.x && y < a.y);};
__bool operator== (Pt a) const { return x == a.x and y ==
a.y;};
__friend T ori(pt a, pt b, pt c) { return (b - a) ^ (c - a
); }
__friend T abs2(pt a) { return a * a; }
};
// using numbers::pi; // c++20
const ld pi = acos(-1);
const ld eps = 1e-8L;
using Pt = pt<ld>;
int sgn(ld x) { return (x > -eps) - (x < eps); } // dcmp
== sgn
ld abs(Pt a) { return sqrt(abs2(a)); }
ld arg(Pt x) { return atan2(x.y, x.x); }
bool argcmp(Pt a, Pt b) { // arg(a) < arg(b)
int f = (Pt{a.y, -a.x} > Pt{}) ? 1 : -1 * (a != Pt{});
int g = (Pt{b.y, -b.x} > Pt{}) ? 1 : -1 * (b != Pt{});
return f == g ? (a ^ b) > 0 : f < g;
}
Pt unit(Pt x) { return x / abs(x); }
Pt rotate(Pt u) { // pi / 2
return {-u.y, u.x};
}
Pt rotate(Pt u, ld a) {
Pt v{sin(a), cos(a)};
return {u ^ v, u * v};
}

istream &operator>>(istream &s, Pt &a) { return s >> a.x
>> a.y; }
ostream &operator<<(ostream &s, Pt &a) { return s << "("
<< a.x << ", " << a.y << ")"<< "; }

bool collinearity(Pt a, Pt b, Pt c) { // 三點共線
return ((b - a) ^ (c - a)) == 0;
}

```

7.2 Line

```

struct Line {
Pt a, b;
Pt dir() { return b - a; }
};
int PtSide(Pt p, Line l) {
return sgn(ori(l.a, l.b, p));
}
bool PtOnSeg(Pt p, Line l) {
return PtSide(p, l) == 0 and sgn((p - l.a) * (p - l.b)
) <= 0;
}
Pt proj(Pt p, Line l) {
Pt dir = unit(l.b - l.a);
return l.a + dir * (dir * (p - l.a));
}

```

7.3 Circle

```

struct Cir {
Pt o;
ld r;
};
bool disjunct(const Cir &a, const Cir &b) { // 分開 // 外
切 -> 1
return sgn(abs(a.o - b.o) - a.r - b.r) >= 0;
}
bool contain(const Cir &a, const Cir &b) {
return sgn(a.r - b.r - abs(a.o - b.o)) >= 0;
}

```

7.4 點線距

```

ld PtSegDist(Pt p, Line l) {
ld ans = min(abs(p - l.a), abs(p - l.b));
if (sgn(abs(l.a - l.b)) == 0) return ans;
if (sgn((l.a - l.b) * (p - l.b)) < 0) return ans;
if (sgn((l.b - l.a) * (p - l.a)) < 0) return ans;
return min(ans, abs(ori(p, l.a, l.b)) / abs(l.a - l.b)
);
}
ld SegDist(Line l, Line m) {
return PtSegDist({0, 0}, {l.a - m.a, l.b - m.b});
}

```

7.5 PointInPolygon

```

int inPoly(Pt p, const vector<Pt> &P) {
const int n = P.size();
int cnt = 0;
for (int i = 0; i < n; i++) {
Pt a = P[i], b = P[(i + 1) % n];
if (PtOnSeg(p, {a, b})) return 1; // on edge
if ((sgn(a.y - p.y) == 1) ^ (sgn(b.y - p.y) == 1))
cnt += sgn(ori(a, b, p));
}
return cnt == 0 ? 0 : 2; // out, in
}

```

7.6 Intersection Of Line

```

bool isInter(Line l, Line m) {
if (PtOnSeg(m.a, l) or PtOnSeg(m.b, l) or
PtOnSeg(l.a, m) or PtOnSeg(l.b, m))
return true;
return PtSide(m.a, l) * PtSide(m.b, l) < 0 and
PtSide(l.a, m) * PtSide(l.b, m) < 0;
}
Pt LineInter(Line l, Line m) {
double s = ori(m.a, m.b, l.a), t = ori(m.a, m.b, l.b);
return (l.b * s - l.a * t) / (s - t);
}
bool isInter(Line l, Line m) {
if (PtOnSeg(m.a, l) or PtOnSeg(m.b, l) or
PtOnSeg(l.a, m) or PtOnSeg(l.b, m))
return true;
return PtSide(m.a, l) * PtSide(m.b, l) < 0 and
PtSide(l.a, m) * PtSide(l.b, m) < 0;
}
Pt LineInter(Line l, Line m) {
double s = ori(m.a, m.b, l.a), t = ori(m.a, m.b, l.b);
return (l.b * s - l.a * t) / (s - t);
}

```

7.7 Intersection Of Circle

```

vector<Pt> CircleInter(Cir a, Cir b) {
double d2 = abs2(a.o - b.o), d = sqrt(d2);
if (d < max(a.r, b.r) - min(a.r, b.r) || d > a.r + b.r
) return {};
Pt u = (a.o + b.o) / 2 + (a.o - b.o) * ((b.r * b.r - a
.r * a.r) / (2 * d2));
double A = sqrt((a.r + b.r + d) * (a.r - b.r + d) * (a
.r + b.r - d) * (-a.r + b.r + d));
Pt v = rotate(b.o - a.o) * A / (2 * d2);
if (sgn(v.x) == 0 and sgn(v.y) == 0) return {u};
return {u - v, u + v}; // counter clockwise of a
}

```

7.8 Intersection Of Circle and Line

```

vector<Pt> CircleLineInter(Cir c, Line l) {
Pt H = proj(c.o, l);
Pt dir = unit(l.b - l.a);
double h = abs(H - c.o);
if (sgn(h - c.r) > 0) return {};
double d = sqrt(max((double)0., c.r * c.r - h * h));
if (sgn(d) == 0) return {H};
return {H - dir * d, H + dir * d};
// Counterclockwise
}

```

7.9 Convex Hull

```

vector<Pt> Hull(vector<Pt> P) {
sort(all(P));
P.erase(unique(all(P)), P.end());
P.insert(P.end(), P.rbegin() + 1, P.rend());
vector<Pt> stk;
for (auto p : P) {
auto it = stk.rbegin();
while (stk.rend() - it >= 2 and \
ori(*next(it), *it, p) <= 0 and \
(*next(it) < *it) == (*it < p)) {
it++;
}
stk.resize(stk.rend() - it);
stk.push_back(p);
}
stk.pop_back();
}

```

```
    return stk;
}
```

7.10 旋轉卡尺

```
auto RotatingCalipers(const vector<Pt> &hull) { // 最遠點
    對 回傳距離平方
    int n = hull.size();
    auto ret = abs2(hull[0]);
    ret = 0;
    if (hull.size() <= 2) return abs2(hull[0] - hull[1]);
    for (int i = 0, j = 2; i < n; i++) {
        Pt a = hull[i], b = hull[(i + 1) % n];
        while(ori(hull[j], a, b) <
            (ori(hull[(j + 1) % n], a, b)))
            j = (j + 1) % n;
        chmax(ret, abs2(a - hull[j]));
        chmax(ret, abs2(b - hull[j]));
    }
    return ret;
}
```

7.11 Convex Hull Trick

```
struct Convex {
    int n;
    vector<Pt> A, V, L, U;
    Convex(const vector<Pt> &A) : A(A), n(A.size()) {
        // n >= 3
        auto it = max_element(all(A));
        L.assign(A.begin(), it + 1);
        U.assign(it, A.end()), U.push_back(A[0]);
        for (int i = 0; i < n; i++) {
            V.push_back(A[(i + 1) % n] - A[i]);
        }
    }
    int inside(Pt p, const vector<Pt> &h, auto f) {
        auto it = lower_bound(all(h), p, f);
        if (it == h.end()) return 0;
        if (it == h.begin()) return p == *it;
        return 1 - sgn(ori(*prev(it), p, *it));
    }
    // 0: out, 1: on, 2: in
    int inside(Pt p) {
        return min(inside(p, L, less{}), inside(p, U,
            greater{}));
    }
    static bool cmp(Pt a, Pt b) { return sgn(a ^ b) > 0; }
    // A[i] is a far/closer tangent point
    int tangent(Pt v, bool close = true) {
        assert(v != Pt{});
        auto l = V.begin(), r = V.begin() + L.size() - 1;
        if (v < Pt{}) l = r, r = V.end();
        if (close) return (lower_bound(l, r, v, cmp) - V.
            begin()) % n;
        return (upper_bound(l, r, v, cmp) - V.begin()) % n;
    }
    // closer tangent point array[0] -> array[1] 順時針
    array<int, 2> tangent2(Pt p) {
        array<int, 2> t{-1, -1};
        if (inside(p) == 2) return t;
        if (auto it = lower_bound(all(L), p); it != L.end()
            () and p == *it) {
            int s = it - L.begin();
            return {(s + 1) % n, (s - 1 + n) % n};
        }
        if (auto it = lower_bound(all(U), p, greater{});
            it != U.end() and p == *it) {
            int s = it - U.begin() + L.size() - 1;
            return {(s + 1) % n, (s - 1 + n) % n};
        }
        for (int i = 0; i != t[0]; i = tangent((A[t[0] = i
            ] - p), 0));
        for (int i = 0; i != t[1]; i = tangent((p - A[t[1]
            = i]), 1));
        return t;
    }
    int find(int l, int r, Line L) {
        if (r < l) r += n;
        int s = PtSide(A[l % n], L);
        return *ranges::partition_point(views::iota(l, r),
            [&](int m) {
                return PtSide(A[m % n], L) == s;
            });
    }
};
```

```
    }) - 1;
};
// Line A_x A_x+1 interset with L
vector<int> intersect(Line L) {
    int l = tangent(L.a - L.b), r = tangent(L.b - L.a)
    ;
    if (PtSide(A[l], L) * PtSide(A[r], L) >= 0) return
        {};
    return {find(l, r, L) % n, find(r, l, L) % n};
}
};
```

7.12 Area Of Circle and Polygon

```
double CirclePoly(Cir C, const vector<Pt> &P) {
    auto arg = [&](Pt p, Pt q) { return atan2(p ^ q, p * q
        ); };
    double r2 = C.r * C.r / 2;
    auto tri = [&](Pt p, Pt q) {
        Pt d = q - p;
        auto a = (d * p) / abs2(d), b = (abs2(p) - C.r * C
            .r) / abs2(d);
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a - sqrt(det)), t = min(1., -a +
            sqrt(det));
        if (t < 0 or 1 <= s) return arg(p, q) * r2;
        Pt u = p + d * s, v = p + d * t;
        return arg(p, u) * r2 + (u ^ v) / 2 + arg(v, q) *
            r2;
    };
    double sum = 0.0;
    for (int i = 0; i < P.size(); i++)
        sum += tri(P[i] - C.o, P[(i + 1) % P.size()] - C.o);
    return sum;
}
```

7.13 Area Of Circle and Triangle

```
double CircleTriangle(Pt a, Pt b, double r) {
    if (sgn(abs(a) - r) <= 0 and sgn(abs(b) - r) <= 0) {
        return abs(a ^ b) / 2;
    }
    if (abs(a) > abs(b)) swap(a, b);
    auto I = CircleLineInter({a}, r, {a, b});
    erase_if(I, [&](Pt x) { return !PtOnSeg(x, {a, b}); });
    ;
    if (I.size() == 1) return abs(a ^ I[0]) / 2 +
        SectorArea(I[0], b, r);
    if (I.size() == 2) {
        return SectorArea(a, I[0], r) + SectorArea(I[1], b
            , r) + abs(I[0] ^ I[1]) / 2;
    }
    return SectorArea(a, b, r);
}
```

7.14 Area Of Sector

$$\angle AOB * r^2 / 2$$

```
double Sector(Pt a, Pt b, double r) {
    double theta = atan2(a.y, a.x) - atan2(b.y, b.x);
    while (theta <= 0) theta += 2 * pi;
    while (theta >= 2 * pi) theta -= 2 * pi;
    theta = min(theta, 2 * pi - theta);
    return r * r * theta / 2;
}
```

7.15 切線

```
vector<Line> CircleTangent(Cir c, Pt p) {
    vector<Line> z;
    double d = abs(p - c.o);
    if (sgn(d - c.r) == 0) {
        Pt i = rotate(p - c.o);
        z.push_back({p, p + i});
    } else if (d > c.r) {
        double o = acos(c.r / d);
        Pt i = unit(p - c.o);
        Pt j = rotate(i, o) * c.r;
        Pt k = rotate(i, -o) * c.r;
        z.push_back({c.o + j, p});
        z.push_back({c.o + k, p});
    }
    return z;
}
```

7.16 公切線

```
vector<Line> CircleTangent(Cir c1, Cir c2, int sign1) {
    // sign1 = 1 for outer tang, -1 for inner tang
    vector<Line> ret;
    ld d_sq = abs2(c1.o - c2.o);
    if (sgn(d_sq) == 0) return ret;
    ld d = sqrt(d_sq);
    Pt v = (c2.o - c1.o) / d;
    ld c = (c1.r - sign1 * c2.r) / d;
    if (c * c > 1) return ret;
    ld h = sqrt(max(0.0, 1.0 - c * c));
    for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
        Pt n = Pt(v.x * c - sign2 * h * v.y, v.y * c +
            sign2 * h * v.x);
        Pt p1 = c1.o + n * c1.r;
        Pt p2 = c2.o + n * (c2.r * sign1);
        if (sgn(p1.x - p2.x) == 0 && sgn(p1.y - p2.y) ==
            0)
            p2 = p1 + rotate(c2.o - c1.o);
        ret.push_back({p1, p2});
    }
    return ret;
}
```

7.17 半平面交

```
bool cover(Line L, Line P, Line Q) {
    // PtSide(LineInter(P, Q), L) <= 0 or P, Q parallel
    i128 u = (Q.a - P.a) ^ Q.dir();
    i128 v = P.dir() ^ Q.dir();
    i128 x = P.dir().x * u + (P.a - L.a).x * v;
    i128 y = P.dir().y * u + (P.a - L.a).y * v;
    return sgn(x * L.dir().y - y * L.dir().x) * sgn(v) >=
        0;
}

vector<Line> HPI(vector<Line> P) {
    // Line P.a -> P.b 的逆時針是半平面
    sort(all(P), [&](Line l, Line m) {
        if (argcmp(l.dir(), m.dir())) return true;
        if (argcmp(m.dir(), l.dir())) return false;
        return ori(m.a, m.b, l.a) > 0;
    });
    int n = P.size(), l = 0, r = -1;
    for (int i = 0; i < n; i++) {
        if (i and !argcmp(P[i - 1].dir(), P[i].dir()))
            continue;
        while (l < r and cover(P[i], P[r - 1], P[r])) r--;
        while (l < r and cover(P[i], P[l], P[l + 1])) l++;
        P[++r] = P[i];
    }
    while (l < r and cover(P[l], P[r - 1], P[r])) r--;
    while (l < r and cover(P[r], P[l], P[l + 1])) l++;
    if (r - l <= 1 or !argcmp(P[l].dir(), P[r].dir()))
        return {}; // empty
    if (cover(P[l + 1], P[l], P[r]))
        return {}; // infinity
    return vector(P.begin() + l, P.begin() + r + 1);
}
```

7.18 最小包覆圓

```
Pt Center(Pt a, Pt b, Pt c) {
    Pt x = (a + b) / 2;
    Pt y = (b + c) / 2;
    return LineInter({x, x + rotate(b - a)}, {y, y +
        rotate(c - b)});
}

Cir MEC(vector<Pt> P) {
    mt19937 rng(time(0));
    shuffle(all(P), rng);
    Cir C = {P[0], 0.0};
    for (int i = 0; i < P.size(); i++) {
        if (C.inside(P[i])) continue;
        C = {P[i], 0};
        for (int j = 0; j < i; j++) {
            if (C.inside(P[j])) continue;
            C = {(P[i] + P[j]) / 2, abs(P[i] - P[j]) / 2};
            for (int k = 0; k < j; k++) {
                if (C.inside(P[k])) continue;
                C.o = Center(P[i], P[j], P[k]);
                C.r = abs(C.o - P[i]);
            }
        }
    }
}
```

```
}
return C;
}
```

7.19 圓聯集面積

```
// Area[i] : area covered by at least i circle
// TODO:!!!aaaa!!!
vector<double> CircleUnion(const vector<Cir> &C) {
    const int n = C.size();
    vector<double> Area(n + 1);
    auto check = [&](int i, int j) {
        if (!contain(C[i], C[j]))
            return false;
        return sgn(C[i].r - C[j].r) > 0 or (sgn(C[i].r - C
            [j].r) == 0 and i < j);
    };
    struct Teve {
        double ang; int add; Pt p;
        bool operator<(const Teve &b) { return ang < b.ang
            ; }
    };
    auto ang = [&](Pt p) { return atan2(p.y, p.x); };
    for (int i = 0; i < n; i++) {
        int cov = 1;
        vector<Teve> event;
        for (int j = 0; j < n; j++) if (i != j) {
            if (check(j, i)) cov++;
            else if (!check(i, j) and !disjunct(C[i], C[j
                ])) {
                auto I = CircleInter(C[i], C[j]);
                assert(I.size() == 2);
                double a1 = ang(I[0] - C[i].o), a2 = ang(I
                    [1] - C[i].o);
                event.push_back({a1, 1, I[0]});
                event.push_back({a2, -1, I[1]});
                if (a1 > a2) cov++;
            }
        }
        if (event.empty()) {
            Area[cov] += pi * C[i].r * C[i].r;
            continue;
        }
        sort(all(event));
        event.push_back(event[0]);
        for (int j = 0; j + 1 < event.size(); j++) {
            cov += event[j].add;
            Area[cov] += (event[j].p ^ event[j + 1].p) /
                2.;
            double theta = event[j + 1].ang - event[j].ang
                ;
            if (theta < 0) theta += 2 * pi;
            Area[cov] += (theta - sin(theta)) * C[i].r * C
                [i].r / 2.;
        }
    }
    return Area;
}
```

7.20 多邊形聯集面積

```
// Area[i] : area covered by at least i polygon
vector<double> PolyUnion(const vector<vector<Pt>> &P) {
    const int n = P.size();
    vector<double> Area(n + 1);
    vector<Line> Ls;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < P[i].size(); j++)
            Ls.push_back({P[i][j], P[i][(j + 1) % P[i].
                size()]});
    auto cmp = [&](Line &l, Line &r) {
        Pt u = l.b - l.a, v = r.b - r.a;
        if (argcmp(u, v)) return true;
        if (argcmp(v, u)) return false;
        return PtSide(l.a, r) < 0;
    };
    sort(all(Ls), cmp);
    for (int l = 0, r = 0; l < Ls.size(); l = r) {
        while (r < Ls.size() and !cmp(Ls[l], Ls[r])) r++;
        Line L = Ls[l];
        vector<pair<Pt, int>> event;
        for (auto [c, d] : Ls) {
            if (sgn((L.a - L.b) ^ (c - d)) != 0) {
                int s1 = PtSide(c, L) == 1;

```

```

        int s2 = PtSide(d, L) == 1;
        if (s1 ^ s2) event.emplace_back(LineInter(
            L, {c, d}), s1 ? 1 : -1);
    } else if (PtSide(c, L) == 0 and sgn((L.a - L.
        b) * (c - d)) > 0) {
        event.emplace_back(c, 2);
        event.emplace_back(d, -2);
    }
}
sort(all(event), [&](auto i, auto j) {
    return (L.a - i.ff) * (L.a - L.b) < (L.a - j.
        ff) * (L.a - L.b);
});
int cov = 0, tag = 0;
Pt lst{0, 0};
for (auto [p, s] : event) {
    if (cov >= tag) {
        Area[cov] += lst ^ p;
        Area[cov - tag] -= lst ^ p;
    }
    if (abs(s) == 1) cov += s;
    else tag += s / 2;
    lst = p;
}
}
for (int i = n - 1; i >= 0; i--) Area[i] += Area[i +
    1];
for (int i = 1; i <= n; i++) Area[i] /= 2;
return Area;
};

```

7.21 Pick’s Theory

$$A = i + \frac{b}{2} - 1, \quad 2A = 2i + b - 2, \quad i = A - \frac{b}{2} + 1$$

其中 A 為面積、 b 為圖形內部的點數量、 i 為圖形線上的點數量

```

vector<Pt> p;
ll compute_area(int n) { // A * 2
    ll area = 0;
    for (int i = 1; i < n; i++)
        area += cross(p[i], p[(i + 1) % n], p[0]);
    area = abs(area);
    return area;
}
ll compute_pt_on_line(int n) { // b
    ll pt = 0;
    for (int i = 0; i < n; i++)
        pt += __gcd(abs(p[i].x - p[(i + 1) % n].x),
            abs(p[i].y - p[(i + 1) % n].y));
    return pt; // 這是網路上找的
}
ll compute_inner_pt(int n) { // i
    return (compute_area(n) - compute_pt_on_line(n)) / 2 +
        1;
}
}

```

8 DP

8.1 區間 DP

時間複雜度 $O(n^3)$ ，常見的 n 在 500 以內，至多 1,000。

```

// 區間大小為 1 為 base case
for (int i = 1; i <= n; i++) dp[i][i] = 0;
// 由小區間往大開始轉移
for (int len = 2; len <= n; len++)
    for (int l = 1, r = len; r <= n; l++, r++) {
        dp[l][r] = INF;
        for (int k = l; k < r; k++)
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] +
                n[i] * m[k] * n[r]);
    }
return dp[1][n];

```

8.2 位數 DP

$dp[位數][狀態]$ ， $dp[pos][state]$ ：
 定義為目前位數在前導狀態為 $state$ 的時候的計數
 例如：求數字沒有出現 66 的數量 $1 \sim r$
 $\rightarrow dp[pos][1]$ 可表示計算 pos 個位數在前導出現一個 6 的計數
 $\rightarrow dp[3][1]$ 則計算 6XXX
 模板的 pos 是反過來的，但不影響 (只是用來 dp 記憶用)
 pos: 目前位數
 state: 前導狀態
 lead: 是否有前導 0
 (大部分題目不用但有些數字 e.g. 00146 如果有影響時要考慮)
 limit: 是否窮舉有被 num 限制

```

vector<int> num;
int dp[20][state];
int dfs(int pos, int state, bool lead, bool limit) {
    if (pos == num.size()) { // 有時要根據不同state回傳情況
        return 1;
    }
    if (limit == false && lead == false &&
        dp[pos][state] != -1)
        return dp[pos][state];
    int up = limit ? num[pos] : 9;
    int ans = 0;
    for (int i = 0; i <= up; i++) {
        // 有時要考慮那些狀況要continue
        ans += dfs(pos + 1, state || (check[i] == 2),
            lead && i == 0, limit && i == num[pos]);
    }
    if (limit == false && lead == false)
        dp[pos][state] = ans;
    return ans;
}

```

8.3 SOS DP

```

for (int mask = 0; mask < (1 << N); mask++)
    F[mask] = A[mask];
for (int i = 0; i < N; i++)
    for (int mask = 0; mask < (1 << N); mask++)
        if (mask & (1 << i)) F[mask] += F[mask ^ (1 << i)];

```

9 Sorting

9.1 Merge Sort

```

template <typename T>
void merge_sort(T &arr, T &tmp, int l, int r) {
    // 陣列元素少於 1 直接結束
    if (r <= l + 1) return;

    // 分成左右兩邊處理
    int m = l + (r - l) / 2;
    merge_sort(arr, tmp, l, m); // [l, m)
    merge_sort(arr, tmp, m, r); // [m, r)

    // 合併兩邊的結果
    for (int i = l, li = l, ri = m; i < r; i++) {
        // 只剩下右邊 // 兩邊都有剩，右邊比左邊小
        if (li == m || (ri != r && arr[ri] < arr[li]))
            tmp[i] = arr[ri++];
        // 只剩下左邊 // 兩邊都有剩，左邊比右邊小
        else tmp[i] = arr[li++];
    }

    // 把結果複製回來
    for (int i = l; i < r; i++) arr[i] = tmp[i];
}

```

9.2 Quick Sort

```

template <typename T>
void quick_sort(T arr[], int l, int r) {
    // 陣列元素少於 1 直接結束
    if (r <= l + 1)
        return;

    // 左邊和右邊的指針
    int li = l + 1, ri = r - 1;
    // 判定的基準點
    T pivot = arr[l];

    // TODO: 左邊 <= 基準點 < 右邊
    while (li != ri) {
        // 右邊的指針往左移動，找比基準點小的值
        while (arr[ri] > pivot && li < ri)
            ri--;
        // 左邊的指針往右移動，找比基準點大的值
        while (arr[li] <= pivot && li < ri)
            li++;
        // 當左右指針沒有相遇時，表示不滿足 TODO
        if (li < ri)
            swap(arr[li], arr[ri]);
    }
}

```




10.7 Graph Paper



